

Verifiable Agentic Science on Public Infrastructure

Dannon Baker¹, Marius van den Beek², John Chilton²,
Danielle Callan², Scott Cain², Steven Weaver³,
Sergei L. Kosakovsky Pond^{3,*}, and Anton Nekrutenko^{2,*}

¹Department of Biology, Johns Hopkins University, Baltimore, MD, USA

²Department of Biochemistry and Molecular Biology, Penn State University, University Park, PA, USA

³Institute for Genomics and Evolutionary Medicine, Department of Biology,
Temple University, Philadelphia, PA, USA

*Correspondence: spond@temple.edu, anton@nekrut.org

September 23, 2026

Abstract

Frontier artificial intelligence models promise to accelerate scientific discovery by automating hypothesis formulation, experimental design, and computational code execution across complex biological domains. Yet contemporary agent platforms operate as disembodied reasoning engines in isolated local environments, generating unversioned scripts, overwhelming consumer hardware on large datasets, and confining scientific discovery to proprietary corporate enclosures. We eliminate this computational bottleneck. Here we introduce an open agentic architecture that provides artificial intelligence with a distributed computational body, transforming public Galaxy cyberinfrastructure into an execution surface governed by immutable directed acyclic graph provenance. To direct long-horizon investigations, we develop Orbit, a scientific research workbench that binds human intent, execution decisions, and remote cloud invocations into a version-controlled research notebook. Complementing this workbench, the Workflow Foundry compiles scientific literature into schema-validated workflows, while unprivileged User-Defined Tools allow agents to execute containerized commands across public supercomputing clusters without administrative intervention. By establishing provenance by construction, this infrastructure prevents the hallucination of scientific method, resolving silent gene identifier traps and converting external workflows into reproducible community assets. All software and universal plugins are freely available, ensuring verifiable scientific discovery across the open public commons.

1 Introduction

Scientific inquiry faces a profound epistemological crisis. The rapid emergence of autonomous artificial intelligence agents promises to compress years of computational investigation into hours, allowing language models to formulate hypotheses, select algorithms, and author code across diverse domains of life science [16]. Yet without rigorous computational grounding, this acceleration threatens the foundation of scientific truth. Left unconstrained, frontier models generate persuasive but unobservable computational fiction—a phenomenon we define as the hallucination of method. Agents author bespoke scripts in transient directories, omit software dependency versions, and manipulate raw data through unrecorded ad hoc transformations. The resulting analyses appear mathematically rigorous and biologically sound, yet rest upon ephemeral code fragments that defy replication.

In computational biology, where conclusions depend entirely on software parameterization and execution lineage, unrecorded agency produces post-verifiable science.

The root of this crisis lies in a fundamental structural misspecification: the mind-body problem of artificial intelligence. For the past three years, automated scientific platforms have operated as disembodied reasoning engines. Systems celebrate the ability of foundation models to debate hypotheses, synthesize literature, or write short script sequences within isolated local sandboxes. Yet empirical science cannot occur in a vacuum. A disembodied reasoning engine cannot interact with petabytes of raw sequence archives, cannot schedule jobs across distributed high-performance clusters, and cannot allocate multi-node physical memory. Operating on personal computers, autonomous agents quickly collide with physical reality. Workstations run out of memory during genome assembly, crash under high-throughput read alignment, or silently truncate input files to fit within local constraints. A mind without a computational body is an armchair philosopher, incapable of executing authentic empirical investigations.

This computational limitation exacerbates a dangerous institutional shift: the privatization of scientific discovery. As frontier foundation models demand vast computational resources, high-capability scientific automation is increasingly sequestered inside commercial walled gardens. Proprietary corporate research platforms link foundation models to private compute clusters and closed cloud storage. This enclosure creates an unequal scientific world. Well-funded commercial institutions conduct automated research at scale, while academic laboratories and researchers in resource-limited environments remain stranded on personal workstations. Commercial platforms occasionally provide static session transcripts or rendered cards, but these artifacts offer only the illusion of reproducibility. An independent researcher cannot re-execute a proprietary conversational transcript, cannot inspect its container layers, and cannot run the underlying code on independent infrastructure. Science risks trading open, universal verification for private corporate authority.

Recent automated research platforms illustrate these architectural trade-offs. The AI Scientist demonstrated fully automated machine learning discovery, generating hypotheses, executing code, and authoring complete manuscripts [12]. However, its execution was restricted to toy algorithmic scripts inside isolated local directories, where unconstrained execution led to hallucinated citations, runaway loops, and unverified numerical claims. Automated discovery stalled on computational rigor. Google DeepMind recently introduced Co-Scientist, a multi-agent framework that implements structured debate to generate and evolve biomedical research proposals [11]. While Co-Scientist produced validated therapeutic hypotheses for drug repurposing, the system acts primarily as a literature reasoning engine, yielding descriptive textual plans rather than executable, cloud-backed computational workflows that can process terabyte-scale biological data. The Virtual Lab demonstrated comparable trade-offs [15]. Coordinating specialized agent personas—principal investigators, scientific critics, and bioinformaticians—the platform designed viable SARS-CoV-2 nanobodies with laboratory validation, yet its computational execution relied on bespoke local scripts that lacked a standardized, shareable execution graph. Kosmos addressed session persistence by employing a structured world model across twelve-hour investigations [14]. Nevertheless, its computational runtime remains confined to basic sandboxes that cannot orchestrate multi-node compute schedules or petabyte-scale public archives.

Domain tool orchestration faces identical structural boundaries. Platforms such as Coscientist [2] and ChemCrow [13] established that large language models can orchestrate specialized chemical synthesis hardware, external computational APIs, and safety protocols. They proved the power of tool augmentation. However, these systems were tailored specifically to small-molecule chemistry and laboratory robotics. They do not address the complex data structures of computational genomics—including paired-end sequencing collections, multi-sample variant calls, and distributed pipeline dependencies—across heterogeneous academic and commercial clusters. In parallel, commercial

developer environments provide rapid software engineering capabilities on local files. Yet these platforms remain blind to biological context. While commercial code assistants parse syntax swiftly, they possess no native knowledge of reference genomes, biological format specifications, or high-performance compute allocations across supercomputing clusters. The investigator remains stranded on an isolated laptop with unpinned dependencies and fragmented scripts. Science requires a shared execution fabric.

Public scientific cyberinfrastructure provides the foundation to resolve these dilemmas. For two decades, the Galaxy platform has maintained an open, community-governed computational ecosystem designed to execute distributed scientific workflows across heterogeneous supercomputing clusters [1]. In the agentic era, Galaxy’s graphical web interface becomes secondary. The platform instead emerges as an infrastructural agentic surface. Through standardized programmatic interfaces, Galaxy provides autonomous agents with immediate access to thousands of containerized bioinformatics tools, petabyte-scale reference data tables, and national high-performance compute allocations without requiring administrative intervention for every user command. Immutable directed acyclic graphs track every input dataset, exact tool revision, execution parameter, and intermediate file. Provenance is preserved by construction rather than through retroactive human discipline.

Here we present an open agentic architecture that bridges autonomous reasoning directly to distributed public computing infrastructure. The framework unites four computational components. First, we introduce Orbit, an open-source scientific research workbench that records hypotheses, analytical plans, execution logs, and cloud invocations directly into a living, version-controlled research notebook. Second, we develop the Workflow Foundry, a casting compiler that translates external scientific literature, Nextflow pipelines [4], and Common Workflow Language specifications [3] into schema-validated Galaxy workflows, eliminating language model hallucination through deterministic graph validation. Third, we establish User-Defined Tools, unprivileged containerized definitions that allow agents to register and run custom tools on remote Galaxy compute clusters safely using sandboxed ECMAScript templating. Fourth, we provide production plugins across major agent platforms—including Claude Code, Google Antigravity, and OpenAI Codex—allowing investigators to connect frontier models to public cyberinfrastructure. By grounding autonomous agents in public infrastructure, we replace isolated computational silos with verifiable, reproducible scientific discovery.

2 The Flow: Agent $\rightarrow$ Infrastructure $\rightarrow$ Result

Empirical biological discovery follows a structured journey: from experimental hypothesis to verified computational result. Our architecture implements this journey as a transparent three-stage pipeline (Figure 1). Investigators can use any autonomous agent runtime, including our custom workbench, Orbit, designed specifically with life scientists in mind. These agents draw domain knowledge directly from the Galaxy Model Context Protocol (MCP), curated Galaxy skills, and Workflow Foundry casting rules. Depending on local hardware availability, the agent prototypes and tests analytical logic on subsampled data before launching full-scale execution across Galaxy cyberinfrastructure. All analytical steps record into an immutable research notebook. This record establishes complete provenance by construction.

2.1 Agents and the Orbit Workbench

The architecture is completely agent-agnostic. Researchers are not tied to a single artificial intelligence vendor or proprietary chat interface. Instead, Galaxy can be driven directly by any compliant agent runtime. To facilitate adoption, we distribute open-source plugin packages for

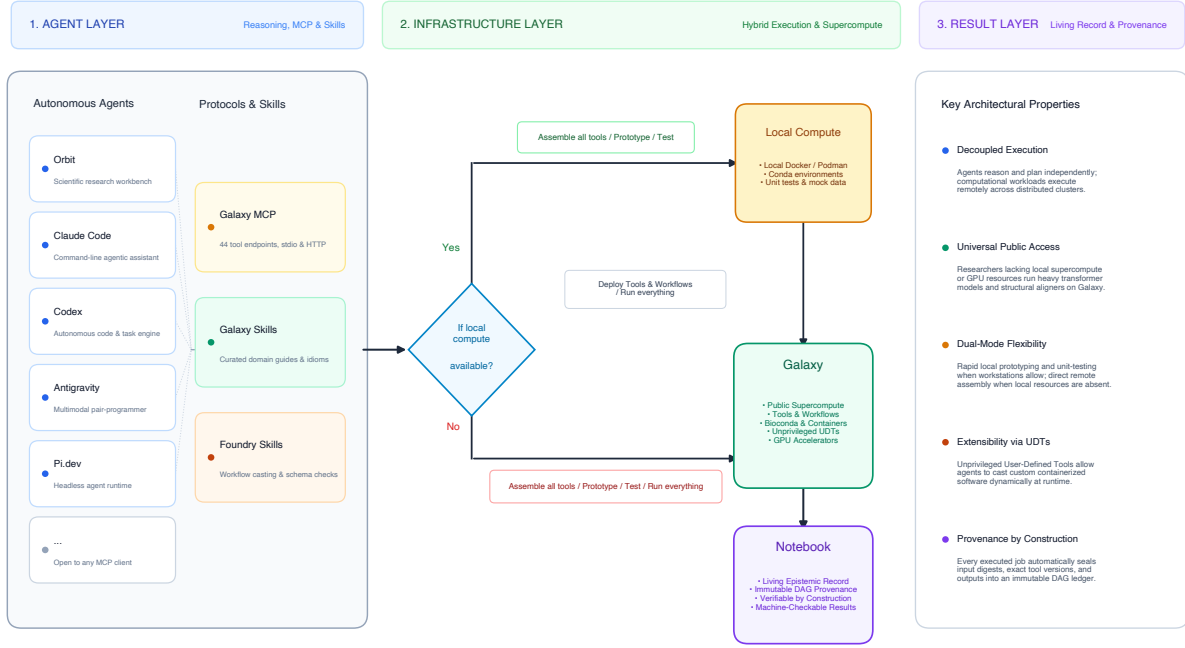


Figure 1: **The flow of verifiable agentic science: from Agent through Infrastructure to Result.** Autonomous agents—including Orbit, Claude Code, OpenAI Codex, Google AntigraVity, and Pi.dev—interact with computational services via the Galaxy Model Context Protocol (MCP), Galaxy skills, and Foundry skills (Agent Layer). Execution branches based on local computational capacity. When local compute is available, agents assemble tools, prototype logic, and run unit tests locally before launching jobs and workflows on Galaxy. When local compute is absent, agents assemble, prototype, test, and execute the complete analytical workflow directly on Galaxy public cyberinfrastructure (Infrastructure Layer). Galaxy executes tasks across distributed supercomputing allocations and GPU clusters via native tools, BioContainers, and unprivileged User-Defined Tools. All analytical operations record directly into a living research notebook. This record preserves an immutable directed acyclic graph with machine-checkable results (Result Layer).

leading agent ecosystems, including Claude Code, OpenAI Codex, Google Antigravity, Cursor, and the Pi runtime [8]. Investigators retain their preferred development environment while connecting directly to public supercomputing resources.

To support biologists directly, we developed Orbit [7]. General-purpose coding assistants assume deep familiarity with command-line interfaces, POSIX shell idiosyncrasies, and cloud environments. In contrast, Orbit provides a dedicated research environment that unifies conversational planning, interactive data visualization, and remote compute orchestration.

Orbit operates flexibly across model providers. Investigators can connect commercial frontier models, including Anthropic Claude, OpenAI GPT, or Google Gemini. Alternatively, researchers handling sensitive clinical data can run private local models through Ollama without transmitting data to external commercial APIs. Orbit incorporates built-in convenience features tailored for life scientists. The workbench manages Galaxy server credentials, polls running cloud jobs asynchronously in the background, renders intermediate datasets, and presents proposed analytical plans for human review before execution begins.

2.2 The Galaxy Model Context Protocol and Skills

Agents interact with computational infrastructure through the Galaxy Model Context Protocol server [5]. The Model Context Protocol establishes an open standard for tool execution and data inspection. Rather than allowing agents to execute arbitrary local shell commands, the server provides forty-four structured endpoints. These endpoints enable agents to search tool registries, inspect dataset collections, launch workflows, and monitor execution states through standard input-output streams or remote HTTP connections.

Domain expertise is provided through modular skill packages [6]. These skills teach language models standard bioinformatics conventions. Skills guide tool discovery, parameter selection, and collection manipulation idioms. For complex multi-step pipelines, Workflow Foundry skills compile validated workflow templates directly into agent memory [10]. By separating operational skills from the underlying compute runtime, the framework ensures that agents execute verified analytical procedures rather than generating unvalidated guesses.

2.3 Local Prototyping and Verification

When local compute resources are available, agents adopt a disciplined, iterative prototyping workflow (Figure 1). On Unix-based operating systems such as macOS and Linux, workstations typically possess container runtimes and native POSIX shells. In contrast, Windows environments often encounter operational friction. Windows systems frequently lack native POSIX environments, suffer line-ending incompatibilities, or restrict container execution when virtualization features are unconfigured. When local resources are available, the agent proceeds through six sequential steps:

1. **Formulate an analytical plan.** The agent outlines the biological objectives, required data transformations, and statistical tests.
2. **Obtain human approval.** The investigator reviews the drafted plan, inspects proposed assumptions, and grants explicit permission to proceed.
3. **Identify software dependencies.** The agent determines which computational tools, libraries, and container images are required.
4. **Prototype on subsampled data.** The agent extracts a representative subset of reads or sequences to test the analytical logic rapidly.

5. **Verify execution integrity.** The agent confirms that commands exit successfully, output files match expected schemas, and quantitative metrics appear biologically sensible.
6. **Map tools to the Galaxy catalog.** The agent surveys the target Galaxy instance to identify which tools are pre-installed in the Tool Shed and which components must be provisioned as User-Defined Tools.

This prototyping loop catches algorithmic flaws early. Testing on small datasets avoids wasting expensive supercomputer allocations on defective scripts.

2.4 Distributed Remote Compute on Galaxy Cyberinfrastructure

Once verified on local data, full-scale analyses transition to Galaxy cyberinfrastructure [1]. Modern Galaxy instances provide immense computational power. Public servers maintain thousands of community-curated, peer-reviewed tools spanning every domain of computational biology. Compute jobs route transparently to national supercomputing centers, high-performance clusters, and specialized hardware accelerators without requiring researchers to manage cluster schedulers. Furthermore, Galaxy maintains petabyte-scale reference data collections, eliminating the need to download large genome indices to local machines.

This infrastructure eliminates a pervasive pathology of autonomous coding agents. Standard agents routinely generate ad hoc Python scripts, store unversioned code in obscure subdirectories, and dump intermediate files into transient temporary directories. Once the agent terminates, nobody knows which code executed, which dependencies were active, or where intermediate data reside. Reproducibility becomes impossible.

Galaxy solves this problem through immutable histories. Every tool execution, exact parameter value, container digest, and intermediate dataset is recorded directly within a persistent directed acyclic graph. Biologists can inspect raw command lines, evaluate standard output logs, view graphical visualizations, and verify every analytical step. However, real-world biological workflows inevitably require custom utilities. Standard tool registries cannot anticipate every bespoke file format converter or novel software utility. For these missing glue steps, the architecture relies on User-Defined Tools.

2.5 Extensibility via User-Defined Tools

User-Defined Tools (UDTs) provide dynamic extensibility on public infrastructure [9]. Introduced in Galaxy 26.1, UDTs allow unprivileged users and autonomous agents to register and run custom tools dynamically. A UDT is declared as a compact YAML specification that pairs an immutable container image with typed inputs and an isolated command template. Because templates evaluate using sandboxed ECMAScript rather than arbitrary host scripts, custom tools execute safely on shared supercomputing clusters.

UDTs span a broad spectrum of computational complexity. At one end, an agent can author a lightweight UDT in seconds to convert unusual tabular formats, filter metadata, or glue disparate workflow steps together. At the opposite end, UDTs orchestrate heavy foundation models requiring dedicated GPU acceleration. Later in this manuscript, we demonstrate this capability in our *Plasmodium vivax* case study. There, the agent authors custom UDTs that route 4.4-gigabyte protein language models directly to NVIDIA A100 GPUs on national supercomputers, executing complex structural alignments without manual system administration.

2.6 Living Research Notebooks and Shared Provenance

Every completed analysis yields two complementary artifacts: an executable Galaxy history and a living research notebook. While the Galaxy history preserves the physical data lineage, the research notebook captures the epistemic narrative. Structured similarly to an `AGENTS.md` record, the notebook documents the complete scientific context: initial hypotheses, biological motivations, human review checkpoints, and quantitative interpretations.

This pairing establishes complete computational provenance. Rather than receiving an isolated PDF report with static figures, independent researchers receive the entire experimental fabric. Galaxy histories, living notebooks, and containerized tool definitions can be published, cited, and shared via persistent URLs. Collaborators can inspect every intermediate dataset, examine why specific parameters were chosen, or re-execute the complete pipeline on public computing resources with a single click. By coupling autonomous agents to open infrastructure, biological discovery becomes fully transparent, verifiable, and reproducible.

3 Empirical Results and Case Studies

To evaluate the operational performance, reproducibility, and computational integrity of this architecture, we conduct three real-world scientific investigations spanning ad hoc empirical reanalysis, literature extraction, and cross-ecosystem pipeline conversion.

3.1 Case Study 1: Heavy-Weight Model Orchestration and Structure-Aware Phylogenomics

Empirical biological discovery frequently requires specialized analytical operations that exceed the boundaries of pre-installed tool registries. Standard agent benchmarks evaluate toy script generation on synthetic problems. In contrast, real-world biological workflows demand complex deep-learning foundational models, structural aligners, and dedicated hardware acceleration. We evaluated our agentic architecture on the structure-aware phylogenomics of the *Plasmodium vivax* PIR multigene family (termed *vir* in *P. vivax*). Operating across three continental reference assemblies, the agent conducted the complete investigation on public cyberinfrastructure (UseGalaxy.org History bbd44e69cb8906b579031c61c5da0079).

Objective and Biological Rationale

The PIR superfamily represents the largest multigene family in the *Plasmodium vivax* genome. VIR surface antigens govern immune evasion, cytoadherence, and red blood cell rosetting (Fig. 2). Because PIR proteins undergo rapid sequence diversification, conventional sequence-based clustering over-splits the family into dozens of artificial subfamilies. Yet crystallographic structures reveal that PIR proteins share a single conserved, mostly- α -helical fold (PDB 6ZYV, Fig. 2e). Sequence-based homology fails within this structural twilight zone.

We sought to determine whether structure-aware phylogenomics unifies the *vir* repertoire across geographically distant isolates. We analyzed three chromosomal long-read reference assemblies: PvP01 from Indonesia, PvW1 from Thailand, and PvPAM from Peru. The study addressed two fundamental questions. First, does structural clustering collapse over-split sequence subfamilies into unified structural classes? Second, does the antigenic repertoire carry geographic lineage structure across continents, or does structural diversity predate continental radiation?

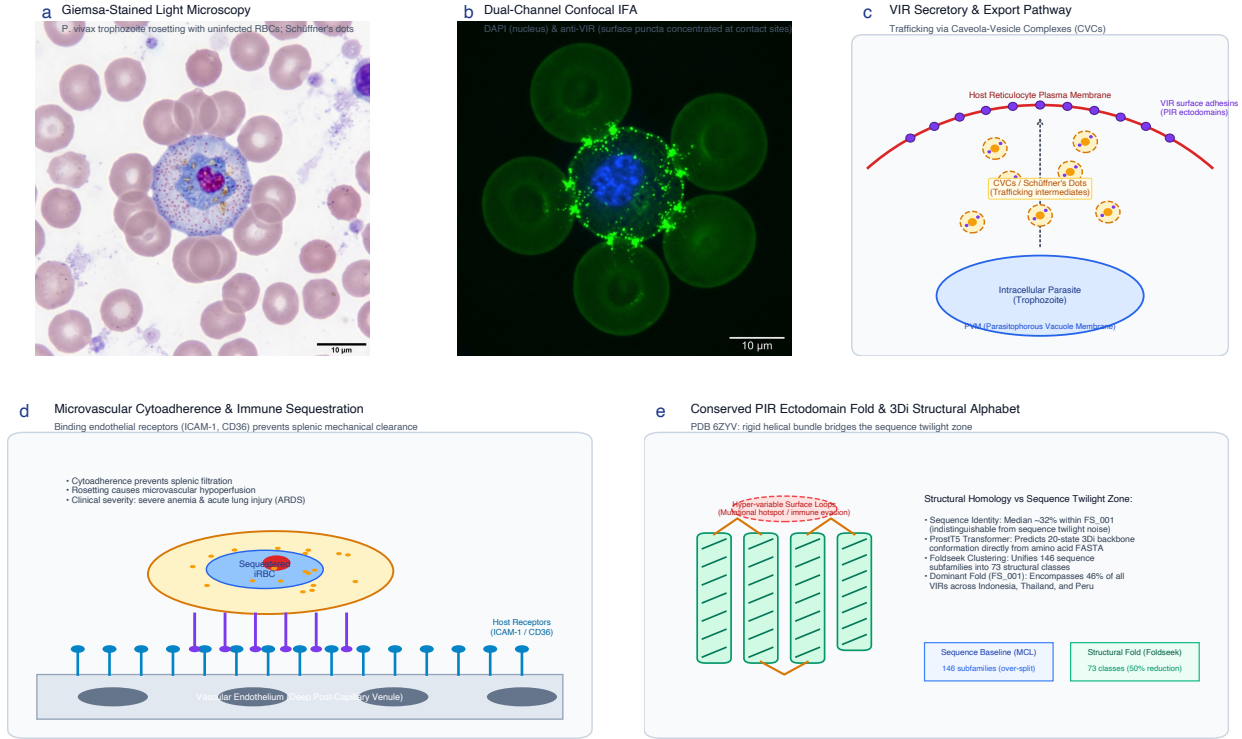


Figure 2: Cellular pathogenesis, microscopy, rosetting, and conserved structural architecture of *Plasmodium vivax* VIR surface antigens. **a**, Brightfield light microscopy of a Giemsa-stained thin blood smear (100 $\times$ oil immersion). An enlarged, polychromatophilic reticulocyte infected with an ameboid *P. vivax* trophozoite forms a multicellular rosette with surrounding uninfected erythrocytes. Characteristic brick-red Schüffner's dots (stippling) pepper the host reticulocyte cytoplasm. Scale bar, 10 μ m. **b**, Dual-channel laser scanning confocal immunofluorescence microscopy (IFA). Parasite nucleus is stained with DAPI (blue). Exported VIR surface proteins are detected with anti-VIR antibodies conjugated to Alexa Fluor 488 (green), displaying punctate membrane localization concentrated at contact junctions with adhering uninfected red blood cells. Scale bar, 10 μ m. **c**, Secretory trafficking pathway. VIR proteins are synthesized by the intracellular trophozoite, cross the parasitophorous vacuole membrane (PVM), and travel through caveola-vesicle complexes (CVCs, corresponding to Schüffner's dots) to the host reticulocyte membrane. **d**, Deep microvascular cytoadherence. Infected reticulocytes adhere to host endothelial receptors (ICAM-1 and CD36) in post-capillary venules, escaping splenic filtration and triggering microvascular hypoperfusion. **e**, Conserved PIR ectodomain structural fold (PDB 6ZYV). A rigid four-helix bundle (α 1- α 4, green) provides structural stability, while hypervariable surface loops (red dashed outline) undergo rapid sequence diversification under humoral immune selection. Predicting 3Di structural alphabets directly from sequences with ProstT5 unifies 89 sequence subfamilies into structural class FS_001.

Heavy-Weight User-Defined Tools and External Model Orchestration

Executing this pipeline required computational capabilities completely absent from standard Tool Shed repositories. Modern structural phylogenomics operates over the structural 3Di alphabet rather than primary amino acids. To avoid the intractable computational cost of predicting three-dimensional coordinates for thousands of proteins, the pipeline predicts 3Di states directly from primary sequences using ProstT5. ProstT5 is a heavy deep-learning transformer model. Running ProstT5 requires 4.4 gigabytes of neural network weights (`prostt5-f16.gguf`), large memory allocations, and dedicated GPU acceleration.

Under standard agent frameworks, this requirement halts analysis. A local agent cannot install a 4.4-gigabyte foundation model on a consumer laptop without exhausting memory or colliding with host CUDA drivers. Commercial cloud runtimes demand private cluster configuration and substantial compute fees.

Our architecture resolved this limitation through unprivileged User-Defined Tools. The agent authored, registered, and executed ten distinct UDT definitions across twenty-seven revisions. To orchestrate heavy computational loads, the agent declared hardware resource requirements directly within the `GalaxyUserTool` definition:

```
requirements: [{type: resource, cores_min: 32, ram_min: 8192, cuda_device_count_min: 1}]
```

Galaxy routed the job through naming conventions. Registering the tool identifier `anton-gpu-xl-foldseek_prostt5_clustering` directed the job to an NVIDIA A100-SXM4-40GB GPU node with CUDA 12.7 and thirty-two CPU cores. For CPU-bound tasks, the agent registered `anton-32-` tools, routing jobs to thirty-two-core compute nodes.

Container resolution presented severe operational hurdles. Initial attempts to run FoldMason failed because upstream containers lacked ProstT5 compilation flags. The agent diagnosed the defect through systematic hypothesis testing, isolating the root cause to the upstream binary. The agent resolved this impasse by implementing a purpose-built container (`quay.io/nekrut/foldmason-prostt5:4.dd3c235-cpu`). To verify execution integrity, the agent instrumented UDTs to emit environment logs containing `nvidia-smi` diagnostics, CUDA driver versions, and allocated thread counts. Heavy foundational models executed remotely on public supercomputers without downloading neural weights or consuming local workstation memory.

Analytical Workflow Flowchart

The investigation progressed through nine analytical modules spanning data ingestion, pseudogene rescue, structural clustering, microsynteny evaluation, and structural phylogenetics (Table 1).

Summary of Empirical Results

The structure-aware investigation yielded five decisive biological and methodological findings:

1. **Structural clustering halves family complexity.** Conventional sequence clustering with MMseqs2 and Markov Clustering partitioned 2,592 VIR proteins into 146 sequence subfamilies. In contrast, Foldseek structural clustering collapsed the family into 73 structural classes. The dominant structural class, designated FS_001, encompassed 1,197 proteins (46% of the total repertoire) and merged 89 independent sequence subfamilies. Structural embeddings unified diversity that sequence comparisons over-split.

Table 1: **Analytical flowchart and execution ledger of the nine-step structure-aware phylogenomics pipeline.** Steps executed on UseGalaxy.org (History bbd44e69cb8906b579031c61c5da0079) through native tools and unprivileged User-Defined Tools.

Step	Module	Tool & UDT Wrapper	Compute ment	Environ-	Primary Data Transforma- tion
1	Ingestion	Native Galaxy Upload	Head node storage		Genomes, proteomes, GFF3s, and PF05795 profile loaded
2	Functional Screen	hmmer_hmmsearch (Tool-Shed)	Standard cluster slot		Profile HMM screen identifies 2,664 candidate VIR proteins
3	Pseudogene Rescue	gff_pseudogene_translator (UDT)	1 core (Python 3.12)		Six-frame translation of GFF3 pseudogenic transcripts
4	Pseudogene Screen	hmmer_hmmsearch (Tool-Shed)	Standard cluster slot		Rescues 75 PvP01 and 70 PvPAM pseudogenized VIR loci
5	Structural Cluster	foldseek_prosttt5 (UDT)	32 cores + NVIDIA A100		4.4 GB ProstT5 model encodes 3Di; clusters 2,592 VIRs
6	Structural MSA	foldmason_align (UDT)	32 cores (custom container)		Lockstep AA and 3Di structural alignment (10,128 columns)
7	Microsynteny	microsynteny_filter (UDT)	8 cores (MMseqs2 + Python)		Filters 194 RBH pairs against 5,120 housekeeping backbone
8	Spatial Profiling	spatial_analysis (UDT)	1 core (Python + SciPy)		Chromosomal coordinate mapping and label-shuffling tests
9	Phylogenomics	iqtree (ToolShed)	32 cores (multithreaded)		Partitioned ML tree inference (AA + 3Di substitution matrix)

2. **Universal structural conservation across continents.** Structural classes proved shared across geography. Over 97.3% of all analyzed VIR proteins belonged to structural clusters containing members from Indonesia, Thailand, and Peru. Lineage-unique clusters accounted for only 2% of sequences, consisting almost entirely of singletons. The structural scaffold of the VIR repertoire is globally conserved.
3. **Absence of geographic phylogeographic signal.** Within the dominant FS_001 cluster, we evaluated 338 sequence-subsampled representatives using maximum-likelihood structural phylogenetics (Fig. 5a). Tip-label parsimony tests treating genome of origin as a discrete character yielded scores indistinguishable from random permutation ($p = 0.55$ for partitioned AA+3Di trees, $p = 0.69$ for AA alone, and $p = 0.26$ for 3Di alone, Fig. 5b). Geographic lineages are interleaved across the tree. Diversification predated continental separation, and ongoing recombination maintains panmictic diversity.
4. **Conserved subtelomeric architecture.** Coordinate analysis revealed that 64.5% of chromosome-assigned VIR loci reside within the outer 10% of chromosomal termini (Fig. 3). Per-chromosome VIR counts correlated tightly across assemblies (Spearman $\rho = 0.909$ to 0.915).
5. **Sequence similarity generates false orthology.** Evaluating 194 reciprocal best-hit pairs between Thailand and Peru within FS_001 revealed that only eleven pairs possessed conserved flanking microsynteny against the housekeeping backbone (Fig. 4). Sequence alignment alone misidentifies orthologs in dynamic multigene families.

Every empirical claim was validated through automated verification scripts, passing seventy-seven independent assertions against raw outputs (77/77 PASS). By coupling autonomous agents to containerized User-Defined Tools and public supercomputers, our architecture executed an advanced structural phylogenomics pipeline without local hardware constraints or unrecorded computational steps.

Biological Summary: Structural Panmixia and Pathogenic Virulence

The findings reshape our understanding of *Plasmodium vivax* pathogenesis and antigenic evolution. Historically characterized as benign tertian malaria, *P. vivax* causes severe clinical disease. Pathology includes acute respiratory distress syndrome, severe anemia, and placental malaria. These syndromes stem directly from the adhesive properties of VIR proteins exported to the host erythrocyte membrane (Fig. 2).

Unlike *P. falciparum*, which infects erythrocytes across all developmental ages, *P. vivax* restricts invasion to young reticulocytes expressing CD71. Following invasion, the ameboid trophozoite remodels the reticulocyte cytoplasm. The parasite establishes caveola-vesicle complexes that appear as classical Schüffner’s dots under light microscopy (Fig. 2a). These membranous structures export VIR adhesins to the host erythrocyte surface (Fig. 2b,c). Once displayed on the membrane, VIR proteins mediate two distinct virulence phenotypes: endothelial cytoadherence and multicellular rosetting.

Rosetting occurs when an infected reticulocyte adheres to multiple uninfected erythrocytes. This interaction forms bulky multicellular clusters in peripheral circulation. These clusters obstruct post-capillary venules and cause microvascular hypoperfusion. Rosetting also shields the parasite from circulating antibodies. Concurrently, surface VIR adhesins bind endothelial receptors including ICAM-1 and CD36 (Fig. 2d). Cytoadherence enables infected cells to sequester in deep vascular beds and the spleen, escaping splenic clearance.

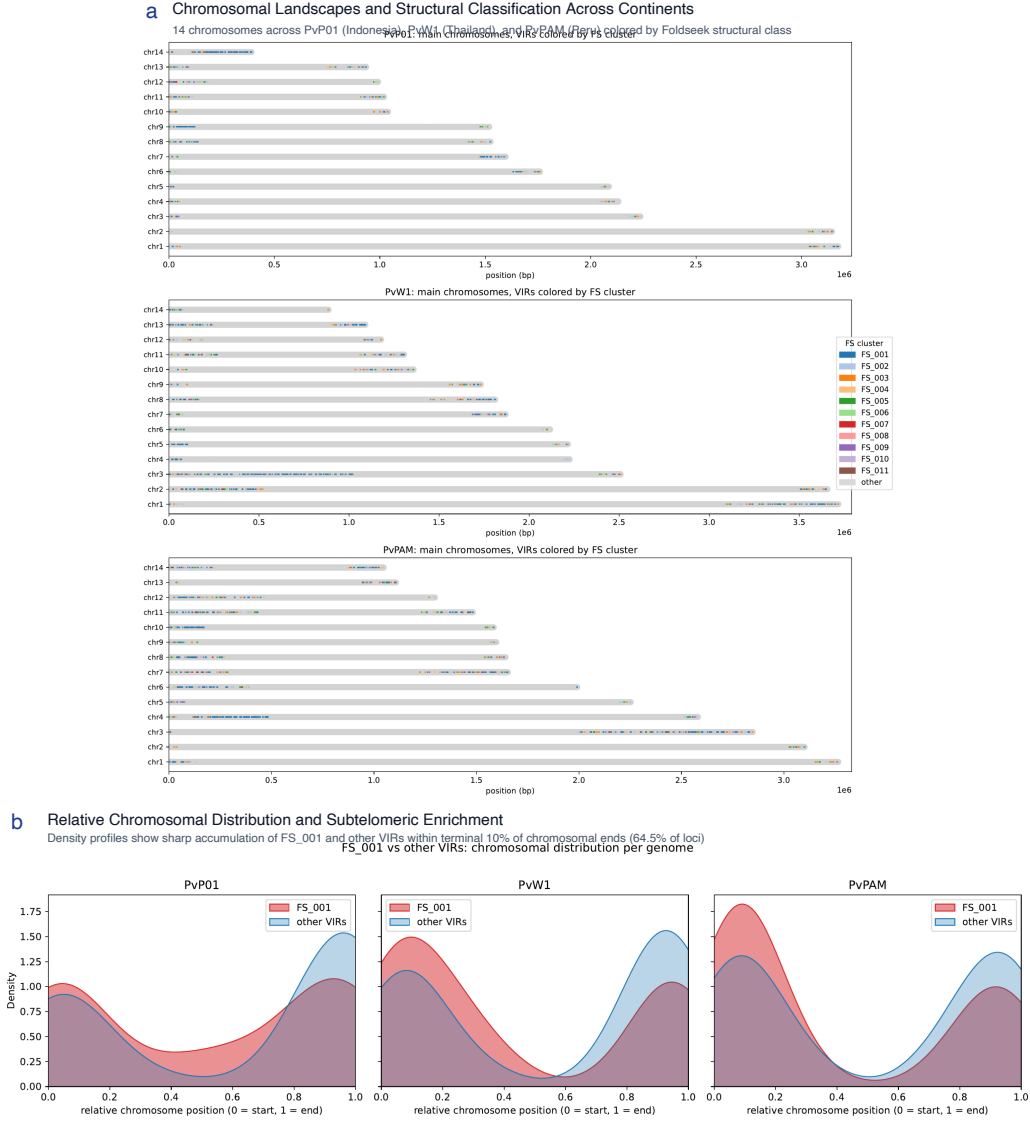


Figure 3: Chromosomal organization and subtelomeric compartmentalization of the VIR multigene family across continental assemblies. **a**, Chromosomal ideograms for all fourteen nuclear chromosomes across three long-read assemblies: PvP01 (Indonesia), PvW1 (Thailand), and PvPAM (Peru). Each tick mark denotes a verified VIR locus, color-coded by its Foldseek structural class assignment. Structural clusters display conserved chromosomal footprints across continents. **b**, Relative chromosomal position density profiles comparing dominant structural class FS_001 against all other VIR clusters. Terminal positions (0.0 and 1.0) represent chromosome ends. Both profiles show pronounced accumulation at chromosomal extremities, with 64.5% of all loci residing within the terminal 10% of chromosomes. Subtelomeric clustering acts as an evolutionary hotspot for ectopic recombination.

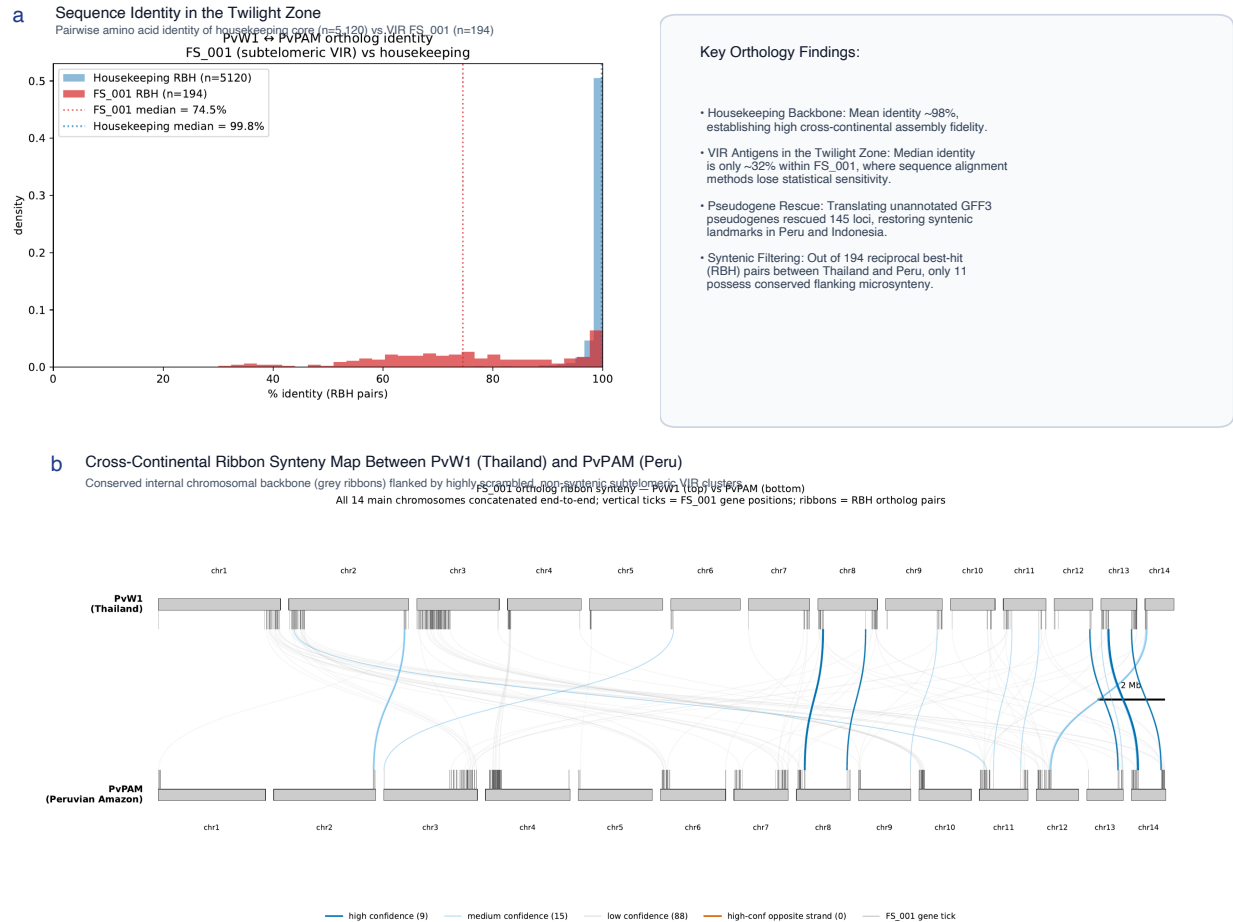
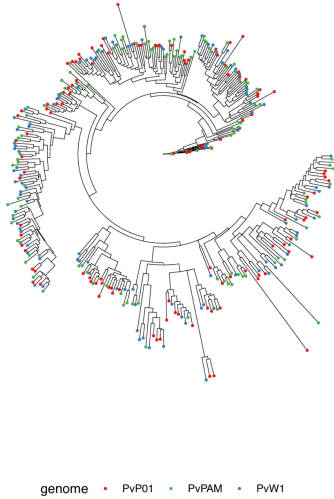


Figure 4: The sequence twilight zone and breakdown of syntenic orthology in multigene antigen families. **a**, Distribution of pairwise amino acid sequence identity between PvW1 (Thailand) and PvPAM (Peru). A reference backbone of 5,120 conserved core housekeeping genes exhibits high sequence identity (mean ~98%, blue), which confirms high assembly accuracy across continents. In contrast, 194 reciprocal best-hit pairs within the dominant VIR structural class FS_001 exhibit a median sequence identity of only 32% (red), residing squarely within the twilight zone where sequence alignment loses sensitivity. **b**, Whole-genome ribbon synteny map between PvW1 (top) and PvPAM (bottom). Grey ribbons denote conserved collinear syntenic blocks in the core chromosome backbone. Subtelomeric regions containing dense VIR clusters exhibit severe syntenic disruption and scrambling. Out of 194 reciprocal best-hit VIR pairs, only eleven maintain conserved flanking microsynteny against the housekeeping backbone.

a Maximum-Likelihood Structural Phylogeny (FoldMason + IQ-TREE)
338 representative sequences from dominant structural class FS_001; tips colored by genome
Q2: FS_001 AA tree (n=338)



b Quantitative Parsimony & Panmixia Analysis
Statistical test of geographic lineage segregation

Continental Tip Distribution:

- PvP01 (Indonesia): 118 tips (34.9%)
- PvW1 (Thailand): 110 tips (32.5%)
- PvPAM (Peru): 110 tips (32.5%)
- Total tips: n = 338 (balanced representation)

Phylogenetic Permutation Tests (1,000 Shuffles):

Model Partition	Obs. Score	Null p-value
Partitioned (AA + 3Di)	160	$p = 0.55$ (NS)
Amino Acid Alone (VT+F+R6)	162	$p = 0.69$ (NS)
3Di Alone (Garg-Hochberg)	156	$p = 0.26$ (NS)

1. Evolutionary Interleaving: Isolates from Asia, South America, and Europe are interspersed throughout the structural tree.
2. Ancient Pre-Radiation Origin: VIR diversity diversified prior to modern continental spread.
3. Global Panmictic Repertoire: Subtelomeric ectopic recombination and gene conversion maintain cross-continental antigenic breadth.

Figure 5: Global structural panmixia and phylogenomics of the dominant VIR fold across continents. **a**, Circular maximum-likelihood structural phylogenetic tree inferred by IQ-TREE 3 from FoldMason multiple structural alignments of 338 sequence-subsampled representatives of dominant structural class FS_001. The tree was inferred using partitioned models combining amino acid substitution matrices (VT+F+R6) and empirical 3Di structural alphabet matrices (Garg-Hochberg). Tip labels are color-coded by reference genome of origin: PvP01 (Indonesia, red), PvW1 (Thailand, green), and PvPAM (Peru, blue). Taxa from Southeast Asia and South America are thoroughly interleaved throughout the topology. **b**, Quantitative tip-label parsimony permutation tests. Observed parsimony scores of genome origin mapped onto the tree (score = 160 for partitioned AA+3Di, 162 for AA alone, and 156 for 3Di alone) do not differ significantly from 1,000 random tip permutations ($p = 0.55$, $p = 0.69$, and $p = 0.26$, respectively). This statistical independence demonstrates that VIR structural diversification preceded the continental divergence of modern *P. vivax*, with ongoing subtelomeric recombination maintaining a panmictic global repertoire.

Immune evasion drives rapid mutation of external surface loops. This selective pressure plunges primary sequence identity into the twilight zone. Within the dominant structural class FS_001, median pairwise sequence identity drops to 32% (Fig. 4a). Sequence alignment tools interpret this divergence as evidence for independent gene families. Consequently, conventional Markov clustering fractures the repertoire into 146 subfamilies. Yet structural modeling reveals that these divergent sequences adopt a single conserved fold (Fig. 2e). The PIR ectodomain consists of a rigid, mostly- α -helical bundle (PDB 6ZYV). By predicting 3Di structural alphabets directly from sequences with ProstT5, Foldseek unifies 89 sequence subfamilies into FS_001, halving the apparent complexity of the repertoire.

This structural stability contrasts with extreme genomic instability. Coordinate mapping demonstrates that 64.5% of VIR loci reside within the outer 10% of chromosomal termini (Fig. 3). While internal chromosomes maintain rigid colinear synteny across continents, subtelomeric domains are completely scrambled (Fig. 4b). Subtelomeric clusters act as an evolutionary recombination engine. High rates of ectopic exchange, unequal crossing over, and gene conversion reshuffle antigenic loops while preserving the core α -helical chassis. Because recombination scrambles flanking genes, sequence similarity between continents produces false orthologs. Out of 194 reciprocal best-hit pairs between Thailand and Peru, only eleven maintain conserved flanking microsynteny.

The maximum-likelihood structural phylogeny resolves an enduring biogeographic debate (Fig. 5a). Prior sequence analyses suggested that modern continental isolates underwent independent, lineage-specific expansions. Under this model, isolates from Indonesia, Thailand, and Peru should segregate into distinct clades. Instead, structural phylogenetics demonstrates complete evolutionary mixing. Taxa from South America and Southeast Asia interleave across the tree. Tip-label parsimony tests confirm that geographic distribution is indistinguishable from random permutation ($p = 0.55$, Fig. 5b). Thus, the structural diversification of VIR antigens predated the continental radiation of modern *P. vivax*. Ongoing subtelomeric recombination maintains a universally shared, panmictic antigenic reservoir across the globe.

3.2 Case Study 2: Autonomous Translation of Published Papers into Workflows

[Case Study 2 Placeholder: Autonomous Literature Conversion]

We evaluate autonomous workflow extraction directly from published scientific literature using the Workflow Foundry. The framework ingests methods sections and maps data flows to Tool Shed wrappers. When software is uncataloged, it synthesizes unprivileged User-Defined Tools. Continuous schema validation yields executable, community-standard `gxformat2` workflows.

3.3 Case Study 3: Cross-Ecosystem Workflow Reusability

[Case Study 3 Placeholder: Nextflow to Galaxy Conversion]

We demonstrate automated cross-ecosystem translation of complex pipelines from Nextflow DSL2 specifications into validated Galaxy workflows. The process maps containers to Bioconda packages, decomposes processes into verified Molds, and assembles test fixtures. This establishes functional equivalence across heterogeneous workflow engines.

4 Discussion

The integration of autonomous artificial intelligence into empirical science represents an epistemological turning point. If autonomous agents continue to operate as disembodied conversationalists inside private local environments, computational science will fragment into an unobservable collection of

irreproducible, post-verifiable claims. By providing artificial intelligence with an open, distributed computational body, our architecture establishes a new paradigm: verifiable agentic science governed by provenance by construction.

4.1 Provenance by Construction

Scientific progress relies on the ability of independent researchers to inspect, challenge, and replicate computational findings. In conventional agent workflows, reproducibility depends entirely on post-hoc human discipline—an assumption that consistently fails in practice. Domain researchers interacting with conversational agents rarely record intermediate commands, pin operating system dependencies, or preserve raw data transformations.

Our architecture transfers the burden of reproducibility from human discipline to computational infrastructure. By executing analyses across Galaxy cyberinfrastructure, every analytical step is recorded upon an immutable directed acyclic graph. Tool versions, container digests, parameter values, and intermediate datasets are sealed in a permanent provenance ledger. This record is not an optional export—it is the physical mechanism through which compute occurs. Consequently, scientific claims generated through this framework carry complete execution lineage, ensuring that autonomous discoveries remain verifiable by the global scientific community.

4.2 Democratizing the Scientific Commons

The computational demands of frontier artificial intelligence threaten to centralize scientific discovery within well-funded commercial institutions. When automated research requires private compute clusters and proprietary cloud platforms, independent academic investigators are systematically excluded from high-throughput discovery.

We demonstrate that public, federally supported cyberinfrastructure can serve as the foundational execution surface for modern artificial intelligence. By pairing open foundation models and universal protocols with the Galaxy platform, we democratize autonomous science. A researcher at an under-resourced institution, equipped only with a standard laptop, can direct autonomous agents that orchestrate distributed supercomputing allocations, query petabyte-scale reference archives, and execute high-throughput pipelines across national high-performance clusters. Preserving the public scientific commons in the era of artificial intelligence is not merely an ethical priority—it is an infrastructural necessity for ensuring universal participation in scientific discovery.

4.3 Toward a Verifiable Scientific Web

Looking forward, coupling autonomous reasoning to open cyberinfrastructure establishes the foundation for a collaborative, planetary-scale scientific network. In this ecosystem, published workflows, curated reference databases, and execution engines communicate through standardized machine-readable protocols. Autonomous agents will not operate as isolated local assistants, but as distributed co-investigators capable of querying global workflow repositories, verifying analytical findings across independent compute clusters, and synthesizing collective biological knowledge. Grounding artificial intelligence in open, verifiable infrastructure ensures that the automated science of the coming decade remains transparent, reproducible, and universally accessible.

Methods

Galaxy Model Context Protocol Implementation

The Galaxy Model Context Protocol server (`galaxy-mcp`) is implemented in Python utilizing the official BioBlend library to communicate with the Galaxy REST API. The server supports two transport mechanisms: standard input-output (stdio) for local CLI integration and streamable HTTP for web and remote clients. Authentication occurs through long-lived Galaxy API keys or OAuth2 token exchanges. The server provides forty-four discrete tool functions mapped to Galaxy endpoints, implementing structured error handling and schema validation for all parameters.

User-Defined Tool Architecture

User-Defined Tools are authored in YAML adhering to the `GalaxyUserTool` class definition. The tool wrapper specifies a mandatory container image hosted on public registries (Quay.io, Docker Hub), typed input parameters, declared output datasets, and an execution command. Commands are templated using sandboxed ECMAScript evaluation executed inside an isolated V8 runtime. Output files are claimed from the working directory through `from_work_dir` designations or dynamic dataset discovery patterns.

Orbit Workbench and Execution Guard

Orbit is constructed as an Electron desktop application powered by a headless agent runtime built as an extension on Pi.dev. The runtime manages session context, system prompts, and tool registration. The execution guard intercepts tool calls via internal lifecycle hooks, classifying actions across model tiers, path security, and shell hazard. Local file writes are strictly confined to the workspace root using realpath resolution. Remote Galaxy jobs are tracked through YAML invocation blocks embedded within `notebook.md`, which are updated via background polling and asynchronous event handlers.

Workflow Foundry Casting Pipeline

The Workflow Foundry compiler is implemented in TypeScript and Node.js. Molds are authored as Markdown documents carrying structured Zod-validated YAML frontmatter declaring pattern dependencies, manual references, and input-output schemas. The casting pipeline compiles Molds into target-specific agent skills. Synthesized workflows are validated statically using the `gxwf` validation engine, which verifies graph topology, step connectivity, and tool state against official Galaxy XML schemas and IWC exemplars.

Data and Code Availability

All software components developed in this work are open-source and publicly available:

- Galaxy MCP Server: <https://github.com/galaxyproject/galaxy-mcp>
- Orbit Workbench and Loom Runtime: <https://github.com/galaxyproject/loom>
- Workflow Foundry: <https://github.com/galaxyproject/foundry>
- Galaxy Skills and Developer Guides: <https://github.com/galaxyproject/galaxy-skills>

- Universal Agentic Plugins: <https://github.com/galaxyproject/agentic-plugins>

All case study datasets, analysis notebooks, and Galaxy history records are accessible via the project repository: <https://github.com/galaxyproject/orbit-paper>.

References

- [1] Linelle Ann L and Abueg, Enis Afgan, Olivier Allart, Ahmed H Awan, Wendi A Bacon, Dannon Baker, Madeline Bassetti, Bérénice Batut, Matthias Bernt, Daniel Blankenberg, Aureliano Bombarely, Anthony Bretaudeau, Catherine J Bromhead, Melissa L Burke, Patrick K Capon, Martin Čech, María Chavero-Díez, John M Chilton, Tyler J Collins, Frederik Coppens, Nate Coraor, Gianmauro Cuccuru, Fabio Cumbo, John Davis, Paul F De Geest, Willem de Koning, Martin Demko, Assunta DeSanto, José Manuel Domínguez Begines, Maria A Doyle, Bert Droesbeke, Anika Erxleben-Eggenhofer, Melanie C Föll, Giulio Formenti, Anne Fouilloux, Rendani Gangazhe, Tanguy Genthon, Jeremy Goecks, Alejandra N Gonzalez Beltran, Nuwan A Goonasekera, Nadia Goué, Timothy J Griffin, Björn A Grüning, Aysam Guerler, Sveinung Gundersen, Ove Johan Ragnar Gustafsson, Christina Hall, Thomas W Harrop, Helge Hecht, Alireza Heidari, Tillman Heisner, Florian Heyl, Saskia Hiltmann, Hans-Rudolf Hotz, Cameron J Hyde, Pratik D Jagtap, Julia Jakiela, James E Johnson, Jayadev Joshi, Marie Jossé, Khaled Jum'ah, Matúš Kalaš, Katarzyna Kamieniecka, Tunc Kayikcioglu, Markus Konkol, Leonid Kostykin, Natalie Kucher, Anup Kumar, Mira Kuntz, Delphine Lariviere, Ross Lazarus, Yvan Le Bras, Gildas Le Corguillé, Justin Lee, Simone Leo, Leandro Liborio, Romane Libouban, David López Tabernero, Lucille Lopez-Delisle, Laila S Los, Alexandru Mahmoud, Igor Makunin, Pierre Marin, Subina Mehta, Winnie Mok, Pablo A Moreno, François Morier-Genoud, Stephen Mosher, Teresa Müller, Engy Nasr, Anton Nekrutenko, Tiffanie M Nelson, Asime J Oba, Alexander Ostrovsky, Polina V Polunina, Krzysztof Poterlowicz, Elliott J Price, Gareth R Price, Helena Rasche, Bryan Raubenolt, Coline Royaux, Luke Sargent, Michelle T Savage, Volodymyr Savchenko, Denys Savchenko, Michael C Schatz, Pauline Segueineau, Beatriz Serrano-Solano, Nicola Soranzo, Sanjay Kumar Srikakulam, Keith Suderman, Anna E Syme, Marco Antonio Tangaro, Jonathan A Tedds, Mehmet Tekman, Wai Cheng (Mike) Thang, Anil S Thanki, Michael Uhl, Marius van den Beek, Deepti Varshney, Jenn Vessio, Pavankumar Videm, Greg Von Kuster, Gregory R Watson, Natalie Whitaker-Allen, Uwe Winter, Martin Wolstencroft, Federico Zambelli, Paul Zierrep, and Rand Zoabi. The galaxy platform for accessible, reproducible, and collaborative data analyses: 2024 update. *Nucleic Acids Research*, 52(W1):W83–W94, May 2024.
- [2] Daniil A. Boiko, Robert MacKnight, Ben Kline, and Gabe Gomes. Autonomous chemical research with large language models. *Nature*, 624(7992):570–578, December 2023.
- [3] Michael R. Crusoe, Sanne Abeln, Alexandru Iosup, Peter Amstutz, John Chilton, Nebojša Tijanić, Hervé Ménager, Stian Soiland-Reyes, Bogdan Gavrilović, Carole Goble, and The CWL Community. Methods included: standardizing computational reuse and portability with the common workflow language. *Communications of the ACM*, 65(6):54–63, May 2022.
- [4] Paolo Di Tommaso, Maria Chatzou, Evan W Floden, Pablo Prieto Barja, Emilio Palumbo, and Cedric Notredame. Nextflow enables reproducible computational workflows. *Nature Biotechnology*, 35(4):316–319, April 2017.
- [5] Galaxy Project. Galaxy Model Context Protocol (MCP) server. <https://github.com/galaxyproject/galaxy-mcp>, 2026. GitHub repository.

- [6] Galaxy Project. Galaxy skills: Modular domain knowledge for autonomous agents. <https://github.com/galaxyproject/galaxy-skills>, 2026. GitHub repository.
- [7] Galaxy Project. Orbit: A scientific research workbench for autonomous biological discovery. <https://github.com/galaxyproject/loom>, 2026. GitHub repository.
- [8] Galaxy Project. Universal Agentic Plugins for Galaxy. <https://github.com/galaxyproject/agentic-plugins>, 2026. GitHub repository.
- [9] Galaxy Project. User-Defined tools (UDT) authoring specification. https://docs.galaxyproject.org/en/release_26.1/dev/user_defined_tools_authoring.html, 2026. Galaxy Release 26.1 Documentation.
- [10] Galaxy Project. Workflow Foundry: Compiling scientific workflows from literature and pipelines. <https://github.com/galaxyproject/foundry>, 2026. GitHub repository.
- [11] Juraj Gottweis, Wei-Hung Weng, Alexander Daryin, Tao Tu, Petar Sirkovic, Artiom Myaskovsky, Grzegorz Glowaty, Felix Weissenberger, Alessio Orlandi, Dan Popovici, Anil Palepu, Keran Rong, Ryutaro Tanno, Khaled Saab, Fan Zhang, Jacob Blum, Andrew Carroll, Kavita Kulkarni, Nenad Tomašev, Dina Zverinski, Ivor Rendulic, Elahe Vedadi, Florian Hasler, Luka Rimanic, Marina Boia, Ivan Budiselic, Ben Feinstein, Mathias Bellaiche, Tom Sheffer, Jan Freyberg, Jeremy Ratcliff, Ottavia Bertolli, Katherine Chou, Avinatan Hassidim, Burak Gokturk, Amin Vahdat, Yuan Guan, Vikram Dhillon, Eeshit Dhaval Vaishnav, Byron Lee, Tiago R. D. Costa, José R. Penadés, Gary Peltz, Yossi Matias, James Manyika, Demis Hassabis, Yunhan Xu, Pushmeet Kohli, Annalisa Pawlosky, Alan Karthikesalingam, and Vivek Natarajan. Accelerating scientific discovery with co-scientist. *Nature*, 655(8122):487–496, May 2026.
- [12] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The AI scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*, 2024.
- [13] Andres M. Bran, Sam Cox, Oliver Schilter, Carlo Baldassari, Andrew D. White, and Philippe Schwaller. Augmenting large language models with chemistry tools. *Nature Machine Intelligence*, 6(5):525–535, May 2024.
- [14] Bodhisattwa Prasad Majumder et al. Kosmos: An AI scientist for autonomous discovery. *arXiv preprint arXiv:2511.02824*, 2025.
- [15] Kyle Swanson, Wesley Wu, Nash L. Bulaong, John E. Pak, and James Zou. The virtual lab of ai agents designs new sars-cov-2 nanobodies. *Nature*, 646(8085):716–723, 2025.
- [16] Xiangru Tang, Bill Qian, Rick Gao, Jiakang Chen, Xinyun Chen, and Mark B Gerstein. Biocoder: a benchmark for bioinformatics code generation with large language models. *Bioinformatics*, 40(Supplement_1):i266–i276, 2024.